



The Lazy Developer

11

MODULE

Budgeting & Cost Management

Before we start crunching numbers, it helps to get a bird's-eye view of where money actually flows when you build and run a web app. Once you've got that, you can run "what-if" scenarios, put guardrails around runaway costs, and even optimise your architecture so you get more bang for every pound or dollar you spend.

Table of Contents

01. Where will you spend money? (8 min)
02. Scenario-Planning: Estimating Your Costs: Case Study - CVPolish (8 min)
03. Spotting and Taming Runaway Costs
04. Post-Launch Cost Audit: Cutting the Fat
05. Case Study: thelazydeveloper.org – Where the Costs Go?

Every successful project begins with a clear ledger. Before you spin up servers or sign up for a fancy design tool, you need to know every place you might spend money. Sketching this cost map up front gives you the context to trim or defer expenses later and avoids those heart-stopping surprises when your first cloud bill arrives.

Domains

Your domain is where people find you and what users type into their browsers. A memorable, branded domain (for example, myapp.com) instantly looks more professional than a long subdomain. That said, during early development you can often get away with a free subdomain from Vercel (myapp.vercel.app) or Netlify (myapp.netlify.app).

Cloudflare is your friend! Easy to use, generous free tier, cheap pricing, and added options like DDoS protection, DNSSec, and so much more. It's my go to for managing all my domains.

SSL certificates, the little padlock in the address bar, are non-negotiable. Without HTTPS your site will be flagged as insecure in modern browsers and your users won't trust you with their data. Luckily Let's Encrypt provides certificates at no cost as do platforms like Vercel manage renewal automatically.

Servers

Your front-end assets (HTML, CSS, JavaScript) and back-end logic (APIs, scheduled jobs, databases) each need a home. Picking the right hosting platform affects performance, scale and your wallet.

Front-End Hosting

Platform	Pros	Cons
Vercel	Instant previews on every git push; free SSL; global CDN	Usage-based billing can spike unexpectedly; function timeouts
Netlify	Generous free tier; built-in form handling; edge functions	Cold starts on free tier; limited build minutes
AWS S3 + CloudFront	Virtually unlimited scale; fine-grained cache controls	Complex setup; data egress fees
GitHub Pages	Free for public repos; incredibly simple	Static sites only; no server-side code

A CDN (content delivery network) like CloudFront or Vercel's edge network brings your assets closer to users around the globe. Caching static files aggressively can slash bandwidth costs and speed up your site. Early on you might start on the free tier of Vercel or Netlify, then later switch to a paid CDN plan once committed-use discounts make per-GB fees cheaper. Many websites or apps may not need a CDN. It just depends on if people are going to be pulling content from you or not and you need a reliable place to store all these assets.

Back-End Hosting

Platform	Pros	Cons
Heroku	Zero-config deployments; addons for Postgres, Redis	Free dynos sleep; production dynos can get pricey at scale
Railway	One-click databases; usage-based billing; Git integration	Newer platform; occasional billing quirks
AWS Lambda / EC2	Pay-per-use compute; auto-scaling; vast ecosystem	Requires AWS expertise; Lambda cold starts
DigitalOcean	Predictable droplet pricing; managed databases available	Manual scaling; fewer global regions

Serverless functions (Lambda) let you pay only for execution time, but they can introduce latency when they “cold start.” Fixed-size droplets or dynos give you a predictable monthly bill but cost you even when traffic is low. For an MVP, a small Railway or Heroku dyno is perfect. As you grow, consider reserved instances or container services with savings plans to lower unit costs.

Prompt to try in AI:

“Generate a markdown file of front-end and back-end providers with their free-tier limits, startup costs and bandwidth allowances so we can align what might work best with our requirements defined in our PRD. Go do some web searching to get the latest pricing.”

Database & Auth

Your users’ data like profiles, preferences, content must live somewhere. Managed solutions like Supabase, Firebase or MongoDB Atlas bundle storage, real-time sync and authentication into one package. They often include a generous free tier (for example, 500 MB storage and 10 000 monthly active users) but then charge per additional GB stored or per 10 000 reads and writes.

Two cost factors to watch: the amount of data you store and the number of operations you perform. Egress fees for pulling data out of your database or storage can also sneak up on you. For tiny MVPs you might even get away with SQLite behind a serverless function or static JSON files if you don’t need real-time updates. At a minimum, Supabase will cost \$10 per instance, and you’ll likely need a dev and prod instance post MVP stage.

Marketing & Email

Getting and keeping users engaged means sending newsletters, drip campaigns and transactional emails (password resets, receipts). Platforms like Brevo and Mailchimp charge by the number of contacts or emails sent are often free up to 1 000 contacts, then £0.001–£0.003 per email beyond. Brevo does have a good free tier though which includes 300 emails a day. In addition there are some good CRM features and it’s easy to integrate into your code with the use of a single API key put into your environment variables.

If you only send a handful of transactional emails, Brevo works great for this as they will provide you with an SMTP server in that free tier too!

Design Tools

High-fidelity mockups in Figma or Framer speed development. But each seat can cost £12–£20 per month once you add collaboration features. Early on you can sketch in free tools like Penpot or even Google Slides. As your UI stabilises, you might cancel extra seats or switch to a cheaper plan. Canva even has a free plan that will help you design some basic graphics. The point is options are out there to do this on a budget! Then you have free icon libraries like Shadcn that can be used in your code which Cursor will know about.

AI & LLM Tokens

If your app invokes large language models the token tally translates directly into cash. Base models can run at around £0.002 per 1 000 tokens while advanced variants cost £0.02 or more for the same volume. Implement prompt caching to avoid duplicate calls and batch related requests whenever possible. During development you can lean on smaller, local open-source LLMs to validate prompts before switching to a paid API. Deepseek is incredibly good value for what you get. If you are really savvy you can run the model yourself if you have the right hardware!

Third-Party APIs and Automation Tools

Anything you don't build yourself like payment gateways (Stripe), geocoding (Google Maps), SMS (Twilio), data scraping (Apify) usually operates on pay-as-you-go pricing. A typical SMS API might charge £0.05 per message while a scraping run on Apify could be £0.01. Always develop on sandbox or free tiers, then swap to production keys at launch. If volume surges, look for volume-discount thresholds or open-source self-hosted replacements.

The same goes for Automation tools like make.com or n8n.org. They are nice free tiers which may be enough for what you need or a great for the MVP. Beyond that though you will need to study their pricing structure. N8N is unique in that you can host it yourself on your own server and run your own automations that way! If you can pay for the hardware you don't have to pay per transaction!

Storage

Object storage platforms (AWS S3, Google Cloud Storage, Supabase Storage) charge per GB stored plus fees for data retrieval and egress. Cold storage (archival) is cheaper per GB but slower and incurs retrieval charges. Hot storage keeps frequently accessed files ready but costs more. Clean up unused assets regularly, compress images and enable caching headers to keep your storage bill under control.

There is a LOT you can have for free but some things you must pay for. With the right analysis using AI to help you figure this out, you will be well on your way to finding the balanced approach. Remember, it's not a bad thing to have usage costs if that means you've got lots of paying customers!

2. Scenario-Planning: Estimating Your Monthly Costs

Let’s make this concrete with an example: you’ve built CVPolish, an MVP that wraps Deepseek’s LLM to rewrite two-page CVs into polished, professional resumes. Every “rewrite” is one call to Deepseek-chat. This month, Deepseek’s cache-miss pricing is roughly:

- Input: \$0.27 per 1 000 000 tokens
- Output: \$1.10 per 1 000 000 tokens

A typical two-page CV is about 1 500 tokens on input and you’ll get roughly 1 500 tokens back once it’s rewritten so that’s 3 000 tokens total. Let’s get aggressive here and assume that on average the user base makes one use of CVPolish per day to tailor to a job. So, on average that’s 30 polishes a month.

Per-rewrite cost (cache miss)

$$3\,000 / 1\,000\,000 \times (\$0.27 + \$1.10) = 0.003 \times \$1.37 = \$0.00411$$

Now, if we take the \$0.00411 and multiply by 30, the cost per user, per month for DeepSeek is now \$0.1233.

2.1 Token-Cost by User Base

Users	Monthly Calls (30 each)	Total Token-Cost
10	300	$300 \times \$0.00411 = \1.23
100	3 000	$3\,000 \times \$0.00411 = \12.33
1 000	30 000	$30\,000 \times \$0.00411 = \123.30
10 000	300 000	$300\,000 \times \$0.00411 = \$1\,233.00$

2.2 “What If?” Spikes on Hacker News

A viral feature can multiply usage briefly. Let’s stress-test the 10 000-user tier with 5x and 10x traffic for one month, it’s unlikely, but for the sake of understanding the scale:

Spike	Effective Calls	Token-Cost
Normal	300 000	\$1 233.00
5x Spike	1 500 000	$\$1\,233.00 \times 5 = \$6\,165.00$
10x Spike	3 000 000	$\$1\,233.00 \times 10 = \$12\,330.00$

Why it matters: Even a one-month 10x spike could burn over \$12 000 in token fees alone.

2.3 Hosting Costs that Scale

Your hosting burn isn't flat because it grows as you add users. Let's assume:

- Vercel (front-end) and Railway (back-end + Postgres) both start at \$0/mo each for up to 10 users, then scale roughly linearly:

These are conservative estimates and actual tiers may step up in blocks but they capture the upward trend. You've got storage costs, egress, etc.

2.4 Total Monthly Burn vs. Revenue

You plan to charge \$4.99/month per user. Stripe takes 2.6%, so your financials may look like this.

Users	Token-Cost	Hosting Cost	Total Cost	Net Rev (@ \$4.99/ user)	Profit	Margin
10	\$1.23	\$40	\$41.23	$10 \times \$4.86 = \48.60	\$7.37	15.2%
100	\$12.33	\$80	\$92.33	$100 \times \$4.86 = \486.00	\$393.67	81.0%
1 000	\$123.30	\$160	\$283.30	$1\,000 \times \$4.86 = \$4\,860.00$	\$4 576.70	94.2%
10 000	\$1 233.00	\$400	\$1 633.00	$10\,000 \times \$4.86 = \$48\,600.00$	\$46 967.00	96.6%

Net revenue per user = $\$4.99 \times (1 - 0.026)$ "H" $\$4.86$ (after Stripe's 2.6% fee).

Insights:

- Even with 30 rewrites per user, token fees remain modest relative to subscription revenue.
- Hosting dominates at low user counts but scales predictably.
- By 100 users you're already above an 80% margin, leaving plenty for marketing, support, and tool subscriptions.

2.5 Pricing Takeaways

- \$4.99/month comfortably covers tokens, hosting, plus overhead.
- A viral spike (10x for a day) is a blip at scale but could threaten early-stage budgets so plan a buffer (e.g., \$10 000) in your runway.
- If you wanted tighter margins or a lower entry price (\$1.99–\$2.99), ensure your CAC (Customer Acquisition Cost) and tooling costs stay below \$0.60–\$1.50 per user.

3. Spotting and Taming Runaway Costs

Traffic spikes are thrilling until your cloud bill arrives. An unguarded API loop or a sudden surge in requests can turn growth into a budget disaster. In this section we'll look at the common “cost exploits,” then build in guardrails so that every spike stays within predictable limits. There are protections on both sides, in your app and also in the cloud platforms you leverage. While we focus on your app in this section, be sure to take the time to study what alerts and monitoring tools are available to you on the cloud side as well. Just ask the AI for help!

3.1 Identify “Cost Exploits”

First, audit your code and workflows for places where usage can balloon unexpectedly. Common culprits include:

- Infinite loops or recursive calls. A bug that retries a failed LLM call forever can rack up thousands of token requests in seconds.
- Large file uploads or downloads. Letting users upload multi-hundred-megabyte documents or images can blow past your storage and egress allowances.
- Batch operations without safeguards. A “bulk rewrite” feature that processes dozens of CVs in one go can be handy but also dangerously costly if misused.

Action: Review your most expensive endpoints (LLM calls, database writes, file transfers) and add simple counters or log-level monitoring so you can see when one user or process is responsible for 90% of the usage. Of course if you don't know what those are do an analysis with AI! Have a discussion going through your documentation, the flow of key files, and see wher things can go haywire.

3.2 Rate-Limiting & Throttling at the Edge

Once you know where the danger zones are, you can throttle traffic before it even hits your core systems. We covered rate-limiting in an earlier module so it's good to recap some strategiees here. Edge-level controls are your first line of defense:

Technique	Where to Configure	Benefit
Fixed window rate limit	Cloudflare, AWS API Gateway,	Simple “X calls per minute” caps
Token bucket	Vercel	per IP or key
	NGINX, Express middleware	Burstable allowances with refill
Leaky bucket	Redis-backed middleware	over time
		Smooths out spikes by queueing
Circuit breaker	Application code (e.g. Hystrix	excess requests
	pattern)	Temporarily blocks repeated
		failures or spikes

Example: On Vercel you can add a small middleware snippet that checks a user’s IP or API key against a Redis counter. If they exceed 60 LLM calls per minute, you return a 429 Too Many Requests before any token fee is incurred.

Real World Example

In one of my apps, I built a queue system that allowed me to see what jobs were going into the queue and more importantly not overlaod my APIs as some of them were bound by how much RAM I was getting from the API provider. Each job took up 4GB of RAM and I was given 8GB so I could only run 2 jobs to that API at a time. The global queue system allowed me to simply queue up jobs for this API provider so only 2 run at any given time preventing using more RAM and incurring more costs.

3.3 Real-Time Cost Alerts

Rate-limits catch over-use at the edge, but you’ll also want a watchtower on your overall spend. Set up alerts so you never get surprised by a 10x daily surge:

- 1. Cost API polling. Many platforms (Vercel, Railway, AWS Cost Explorer) offer billing or usage APIs. Ask the AI to plug into them and give you a dashboard with an overview.
- 2. Threshold rules. Choose sensible trigger points e.g. when your month-to-date spend hits 50%, 80% and 95% of budget.
- 3. Notification channels. Push alerts to Slack, email, or SMS so you or your team can jump in immediately. Bonus: Program in some automatic changes to stop the bleeding.

Example Alert Workflow:

3.4 Quotas & Paid-Only Features

Finally, consider hard caps and feature gating so free or trial users can’t drive runaway costs. Let's get back to our app, CVPolish.

Strategy	Use Case	User Impact
Free-tier quotas	Limit free users to 10 rewrites per month	Encourages upgrade or self-service
Per-feature caps	Disable bulk-upload or premium templates	Protects expensive endpoints
Token top-ups	Allow purchase of extra LLM credits	Monetizes heavy users
Tiered APIs keys	Issue different API keys per plan	Tracks usage by plan automatically

Real-World Example: CVPolish might let free users polish up to 10 CVs per month, then require a paid plan for unlimited rewrites. A “pay-as-you-go” top-up option so say \$5 for 1 000 extra rewrites gives power users flexibility while covering your marginal cost.

Don't forget, you can always start off with a controlled beta if you have some funds to put aside to test how the app is working under a smaller load of users. This users who are given access for free will at least give you an idea of how the app is holding up when multiple people are doing things at once. This is something that's a bit harder to test on your own.

4. Post-Launch Cost Audit: Cutting the Fat

Once CVPolish is live and real users are polishing their resumes, you'll have concrete data on which services are earning their keep and which you can slim down or retire altogether. A regular cost audit lets you reclaim budget from under-used add-ons and allocate it where it really drives value. This section is going to seem like the most basic but it's really all common sense and a fresh reminder to check your subscriptions! Are you paying for things you simply don't need anymore or as much of?

4.1 Review Usage Dashboards for Under-Utilized Services

Head into each platform's dashboard like Railway, Supabase, Vercel, and all those design-tool admin panels. Look for services you're paying for but not fully using.

- Database storage vs. reads/writes. Are you paying for a larger storage tier?
 - Authentication seats. If you bought higher grade Supabase hardware tier, can you lower it safely?
 - Design-tool licences. Still using that Canva subscription or can you go down to the free tier now?
 - Add-ons and plugins. CVPolish might have enabled error-tracking or performance monitoring add-ons on every environment. Do you need them on staging or just production?
-

4.2 Downgrade or Cancel Over-Provisioned Tiers

As usage patterns emerge, you can often move down one or two service tiers without impacting performance:

- How to proceed:
-

4.3 Migrate to Self-Hosted or Open-Source Where It Makes Sense

Some paid services are invaluable during early development, but once you've ironed out your workflows, open-source or self-hosted alternatives can save money:

- Example: Error Tracking. Sentry's paid plan may cost \$29/month; migrating to an open-source tool like Exceptionless on a small DigitalOcean droplet could run under \$5/month.
- Example: Email Relay. If you're only sending a couple of hundred transactional emails daily, Brevo let's you send 300 free emails a day.
- Example: Analytics. Swap out a paid SaaS analytics tool for a self-hosted Matomo or Plausible instance once your tracking requirements stabilize. Just run it in Railway next to your other services?

4.4 Re-Evaluate Your LLM Tier other other coding app tiers

Early on, CVPolish leaned heavily on AI assistance so you chose a higher tier for your model of choice (eg. Claude Code Max). Now that the core features are in place and you're comfortable with the platform:

- Check your AI-assist usage. Look at how many hours or credits you actually consumed last month.
- Consider a smaller plan + top-up credits. Maybe you can downgrade now to the basic Pro plan.
- Reassess regularly. As your team grows or development slows, dial your plan up or down to match real needs.

It's all common sense remember! Over the course of a year, a few tweaks and cuts here and there could pay for a nice Christmas party at the end of the year!

7. Case Study: thelazydeveloper.org: A Real-World Breakdown

Let’s pull back the curtain on our own training site, thelazydeveloper.org, to show you how these cost buckets play out in practice. All figures below are approximate monthly costs for our current usage levels.

Service	Role	Monthly Cost (USD)	Notes
Vercel	Front-end hosting	\$20	Front end instance shared with other projects
Railway	Back-end hosting & managed Postgres	\$20	Back end instance share with other projects
Supabase	Real-time database, auth, and file storage	\$20	500GB of storage, auth, and an instance for dev and prod
Cloudflare	DNS management, CDN caching, bot protection	FREE	Basic plan, protects against abusive traffic spikes
hCaptcha	Spam prevention on signup forms	FREE	Low volume, pay-as-you-go on solved captchas
Claude Code	AI-assist credits and Max subscription	\$100	However I use this across multiple projects
Email & Analytics	Brevo for transactional emails; Google for analytics	FREE	Up to 300 emails/day + lightweight analytics
Total		\$160	Of which about \$30-40 may go towards

Why this matters: At \$30/month, we cover all our core infrastructure, AI-assist time, and user-facing services. Scaling to 1 000 users would bump a few lines (Railway and Supabase maybe) but overall costs remain predictable.

Efficiently building this site and use the best practices we discuss has helped. Generous free tiers from various providers helps too.

Recap of the sections:

- 1. Where Will You Spend Money?
- 2. Scenario-Planning: Estimating Your “Runway”
- 3. Spotting and Taming Runaway Costs
- 4. Post-Launch Cost Audit: Cutting the Fat
- 5. The Devil’s in the Details: Egress, Overages & Hidden Fees

With these lessons and real-world numbers, you’re equipped to forecast your burn rate, guard against runaway spend, and design a pricing model that keeps your app both sustainable and profitable.